



PROBLEMAS DE SECCIÓN CRÍTICA

Explicaremos cada uno de los problemas de sección crítica mediante ejemplos.

1. DEADLOCK

Para ejemplificar el problema de deadlock (punto muerto) en el lenguaje Go, las fuentes proporcionan un escenario clásico donde dos goroutines intentan adquirir los mismos recursos en órdenes opuestos, quedando ambas bloqueadas indefinidamente:

Ejemplo de Deadlock con sync.Mutex

En este ejemplo, tenemos un sistema que intenta sumar los valores de dos estructuras. Cada estructura tiene su propio bloqueo (sync.Mutex) para proteger su valor.

```
package main

import (
    "fmt"
    "sync"
    "time"
)

func main() {
    type value struct {
        mu    sync.Mutex
        value int
    }

    var wg sync.WaitGroup

    printSum := func(v1, v2 *value) {
        defer wg.Done()
        v1.mu.Lock() // Bloquea el primer recurso [1]
        defer v1.mu.Unlock()

        time.Sleep(2 * time.Second) // Simula trabajo y facilita el deadlock [3]

        v2.mu.Lock() // Intenta bloquear el segundo recurso [1]
        defer v2.mu.Unlock()

        fmt.Printf("sum=%v\n", v1.value+v2.value)
    }

    var a, b value
    wg.Add(2)
    go printSum(&a, &b) // Goroutine 1 bloquea 'a' e intenta bloquear 'b' [1]
    go printSum(&b, &a) // Goroutine 2 bloquea 'b' e intenta bloquear 'a' [1]
    wg.Wait()
}
```



¿Por qué ocurre el Deadlock?

Al ejecutar este código, el entorno de ejecución de Go detectará que no se puede realizar ningún progreso y emitirá el siguiente error fatal: fatal error: all goroutines are asleep - deadlock!

Este fallo ocurre porque se cumplen las cuatro condiciones de Coffman simultáneamente:

Exclusión Mutua: La función `printSum` requiere derechos exclusivos sobre ambos recursos (a y b) para operar.

Condición de Espera (Wait For): La primera goroutine retiene el bloqueo de a mientras espera obtener el de b. Al mismo tiempo, la segunda goroutine retiene b y espera por a.

Sin Apropiación (No Preemption): Los bloqueos de los mutexes solo pueden ser liberados voluntariamente por la goroutine que los activó; el sistema no puede "quitarles" el recurso.

Espera Circular: Se ha creado una cadena donde la primera goroutine espera a la segunda y la segunda espera a la primera, formando un ciclo que nunca se romperá.

Detección en Go

El runtime de Go es capaz de identificar automáticamente cuando todas las goroutines están bloqueadas (asleep) y el programa no puede recuperarse, terminando el proceso con un mensaje de error explícito para ayudar al desarrollador a identificar el fallo lógico.



2. LIVELOCK

Para ejemplificar el problema de livelock en Go, las fuentes describen una situación en la que los procesos concurrentes están activos y realizando operaciones, pero su estado no progresa hacia una conclusión útil. A diferencia del deadlock, donde los procesos están "congelados", en un livelock los procesos están "ocupados sin hacer nada", un ejemplo clásico es el de dos personas en un pasillo estrecho que intentan dejarse pasar mutuamente, moviéndose de un lado a otro sincronizadamente y bloqueándose repetidamente de forma activa.

Este ejemplo utiliza dos `sync.Mutex` como recursos y el método `TryLock()` (disponible desde Go 1.18) para simular a dos trabajadores "demasiado educados" que, al notar un conflicto, ceden su recurso para intentar evitar bloquear al otro.

```
package main

import (
    "fmt"
    "sync"
    "time"
)

func main() {
    var resA, resB sync.Mutex
    var wg sync.WaitGroup

    // Función que simula un trabajador intentando obtener dos recursos
    worker := func(name string, mu1, mu2 *sync.Mutex) {
        defer wg.Done()
        for {
            mu1.Lock() // Bloquea el primer recurso
            fmt.Printf("%s: Bloqueó Recurso 1. Intentando Recurso 2...\n", name)

            time.Sleep(10 * time.Millisecond) // Simula un pequeño retraso

            // En lugar de esperar eternamente (deadlock), intenta obtener el segundo
            if mu2.TryLock() {
                fmt.Printf("%s: ¡Éxito! Obtuvo ambos recursos.\n", name)
                mu2.Unlock()
                mu1.Unlock()
                return
            }

            // Si el segundo recurso está ocupado, soltamos el primero para ser "amables"
            fmt.Printf("%s: Recurso 2 ocupado, soltando Recurso 1 para evitar bloqueo...\n", name)
            mu1.Unlock()

            // Los trabajadores reintentan casi al mismo tiempo, recreando el conflicto
            time.Sleep(10 * time.Millisecond)
        }
    }

    wg.Add(2)
    // Ambos intentan obtener los recursos en orden opuesto
    go worker("Alicia", &resA, &resB)
    go worker("Beto", &resB, &resA)

    wg.Wait()
}
```



¿Porque ocurre Livelock?

Intento descoordinado de evitar un deadlock: El livelock suele ser el resultado de dos o más procesos que intentan prevenir un punto muerto de forma independiente y sin coordinación. Alicia bloquea el recurso A y al ver que Beto tiene el B, decide soltar el A para "ayudar". Beto hace exactamente lo mismo al mismo tiempo.

Actividad sin progreso real: Aunque ambos están consumiendo CPU y realizando llamadas a funciones (Lock, TryLock, Unlock), el programa se encuentra en un ciclo de transiciones de estado del cual no puede salir. En cada ciclo, ambos regresan al punto de partida sin haber completado su tarea.

Simetría y sincronización: Al reaccionar de la misma manera ante la contención del recurso y reintentar casi al mismo tiempo, los trabajadores recrean el conflicto perpetuamente. Esto se asemeja a la analogía del pasillo donde dos personas se mueven de lado a lado simultáneamente para dejarse pasar, pero terminan bloqueándose de nuevo en cada paso.

Falta de un "árbitro" o prioridad: El problema persiste porque no hay un mecanismo que rompa la simetría (por ejemplo, que uno espere más tiempo que el otro o que uno tenga prioridad sobre el recurso), lo que los mantiene en un juego eterno de "ceder el paso".

Consideraciones Adicionales

Actividad sin progreso: El programa consumirá CPU y verás mensajes en la consola constantemente, pero el estado nunca llegará a la conclusión útil (el return de la función).

Falta de coordinación: El problema surge porque ambos trabajadores intentan prevenir un deadlock de forma independiente; si se coordinaran para que uno se detuviera, el livelock se rompería.

Relación con la Inanición: Este escenario es un subconjunto de la inanición (starvation), donde todos los procesos involucrados son "matados de hambre" por igual al no poder acceder a los recursos que necesitan para terminar su tarea.



3. STARVATION

Para ejemplificar el problema de starvation (inanición) en Go, las fuentes proporcionan un escenario donde una goroutine "codiciosa" acapara un recurso compartido, impidiendo que otra goroutine realice su trabajo de manera eficiente.

Ejemplo de Starvation con sync.Mutex

En este ejemplo, se comparan dos goroutines: un greedyWorker (trabajador codicioso) y un politeWorker (trabajador educado). Ambos compiten por el mismo sync.Mutex para realizar una cantidad idéntica de trabajo simulado.

```
package main

import (
    "fmt"
    "sync"
    "time"
)

func main() {
    var wg sync.WaitGroup
    var sharedLock sync.Mutex
    const runtime = 1 * time.Second

    // Trabajador codicioso: expande su posesión del bloqueo innecesariamente
    greedyWorker := func() {
        defer wg.Done()
        var count int
        for begin := time.Now(); time.Since(begin) <= runtime; {
            sharedLock.Lock()           // Adquiere el bloqueo
            time.Sleep(3 * time.Nanosecond) // Simula trabajo extenso dentro del bloqueo
            sharedLock.Unlock()         // Lo libera
            count++
        }
        fmt.Printf("Trabajador codicioso ejecutó %v ciclos de trabajo\n", count)
    }

    // Trabajador educado: intenta liberar el bloqueo frecuentemente para ser justo
    politeWorker := func() {
        defer wg.Done()
        var count int
        for begin := time.Now(); time.Since(begin) <= runtime; {
            // Divide su trabajo en secciones críticas más pequeñas
            sharedLock.Lock()
            time.Sleep(1 * time.Nanosecond)
            sharedLock.Unlock()

            sharedLock.Lock()
            time.Sleep(1 * time.Nanosecond)
            sharedLock.Unlock()

            sharedLock.Lock()
            time.Sleep(1 * time.Nanosecond)
            sharedLock.Unlock()

            count++
        }
        fmt.Printf("Trabajador educado ejecutó %v ciclos de trabajo\n", count)
    }

    wg.Add(2)
    go greedyWorker()
    go politeWorker()
    wg.Wait()
}
```



¿Por qué ocurre la Inanición?

Acaparamiento de recursos: El trabajador codicioso mantiene el bloqueo compartido durante todo su ciclo de trabajo. Por el contrario, el trabajador educado entra y sale de la sección crítica tres veces por ciclo.

Ineficiencia por justicia: Debido a que el trabajador educado libera y vuelve a solicitar el bloqueo repetidamente, el trabajador codicioso tiene más probabilidades de "ganar" el acceso al recurso cuando queda libre. Como resultado, el trabajador codicioso logra completar casi el doble de trabajo que el educado en el mismo periodo de tiempo.

Detección: Este problema se identifica mediante métricas. Al registrar cuándo se completa el trabajo, se puede observar que el trabajador educado es ineficiente no por su algoritmo, sino porque está siendo "matado de hambre" por el otro proceso.

Consideraciones Adicionales

Recursos afectados: La inanición no solo ocurre con mutexes; también puede afectar el tiempo de CPU, memoria, descriptores de archivos o conexiones a bases de datos.

Solución: Para mitigar la inanición, los desarrolladores deben equilibrar la sincronización de grano grueso (mejor rendimiento individual) con la sincronización de grano fino (mayor equidad entre procesos).